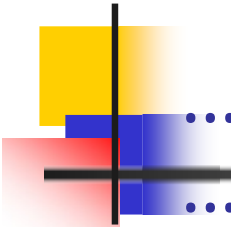




... - Introduction to Transaction Processing

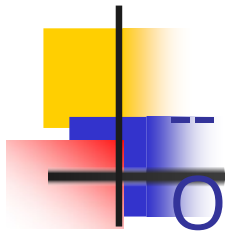
- **A Transaction:** logical unit of database processing that includes one or more access operations (read -retrieval, write - insert or update, delete).
- **A transaction (set of operations)** may be stand-alone specified in a high level language like SQL submitted interactively, or may be embedded within a program.
- **Transaction boundaries:** Begin and End transaction.
- An **application program** may contain several transactions separated by the Begin and End transaction boundaries.



... - Introduction to Transaction Processing

SIMPLE MODEL OF A DATABASE (for purposes of discussing transactions):

- **A database** - collection of named data items
- **Granularity of data** - a field, a record , or a whole disk block (Concepts are independent of granularity)
- Basic operations are **read** and **write**
 - **read_item(X)**: Reads a database item named X into a program variable. To simplify our notation, we assume that *the program variable is also named X*.
 - **write_item(X)**: Writes the value of program variable X into the database item named X.



Read Operation

- Basic unit of data transfer from the disk to the computer main memory is one block. In general, a data item (what is read or written) will be the field of some record in the database, although it may be a larger unit such as a record or even a whole block.
- **read_item(X) command includes the following steps:**
 1. Find the address of the disk block that contains item X.
 2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
 3. Copy item X from the buffer to the program variable named X.



-- Write Operation

- **write_item(X) command includes the following steps:**
 1. Find the address of the disk block that contains item X.
 2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
 3. Copy item X from the program variable named X into its correct location in the buffer.
 4. Store the updated block from the buffer back to disk (either immediately or at some later point in time).



-- A Sample transaction

- Transaction to transfer \$50 from account A to account B :

read($\mathcal{A}$)

$\mathcal{A} := \mathcal{A} - 50$

write($\mathcal{A}$)

read($\mathcal{B}$)

$\mathcal{B} := \mathcal{B} +$
 50

write($\mathcal{B}$)

- Desirable Properties of Transactions

ACID properties:

Atomicity: A transaction is an atomic unit of processing; it is either performed in its entirety or not performed at all.

Simple Example: Bank Transfer

Suppose Ali transfers \$100 from Account A to Account B.

Steps in the transaction

1. Subtract \$100 from Account A
2. Add \$100 to Account B

What Atomicity ensures

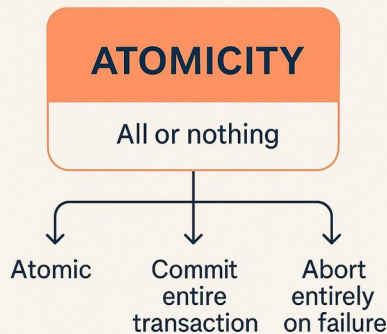
- If both steps succeed → transaction is committed.
- If step 2 fails (e.g., system crash) → step 1 is undone, and Account A gets its \$100 back.

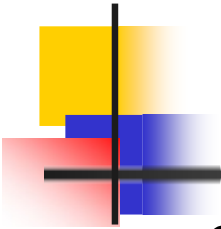
Without Atomicity

Money could disappear from Account A without reaching Account B.

Easy Way to Remember

“All operations happen together, or none happen at all.”





■ **Consistency preservation:** A correct execution of the transaction must take the database from one consistent state to another.

Simple Example: Exam Marks

A university database has a rule:

Marks must be between 0 and 100.

If a transaction tries to store a mark of 150, the database rejects it.

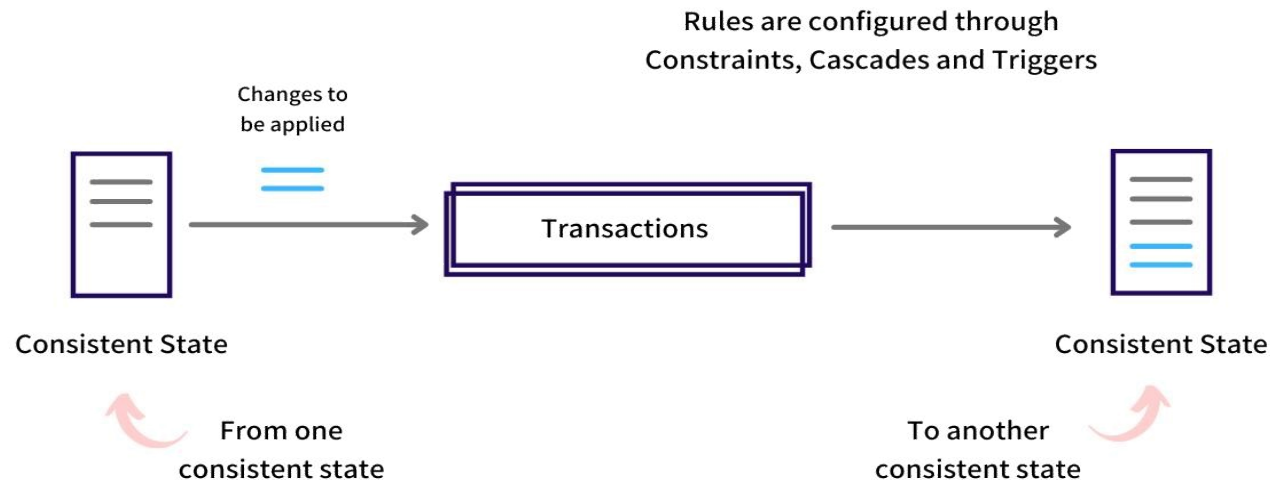
What Consistency ensures

- Data remains accurate and meaningful.

- Rules such as primary keys, foreign keys, and constraints are always maintained.

Without Consistency :-The database could contain invalid or contradictory data.

Easy Way to Remember:- “The database must always remain correct and valid.”



Consistency

- **Isolation:** A transaction should not make its updates visible to other transactions until it is committed; this property, when enforced strictly, solves the temporary update problem and makes cascading rollbacks of transactions unnecessary (see Chapter 21).

Changes made by one transaction should not be visible to other transactions until the transaction is committed. Transactions run as if they are happening one at a time.

Simple Example: Online Shopping

Two customers try to buy the last available laptop at the same time.

What Isolation ensures

- Only one customer can successfully purchase the laptop.
- The second customer sees the updated stock only after the first transaction is committed.

Without Isolation

Both customers might think the laptop is available, causing incorrect sales records.

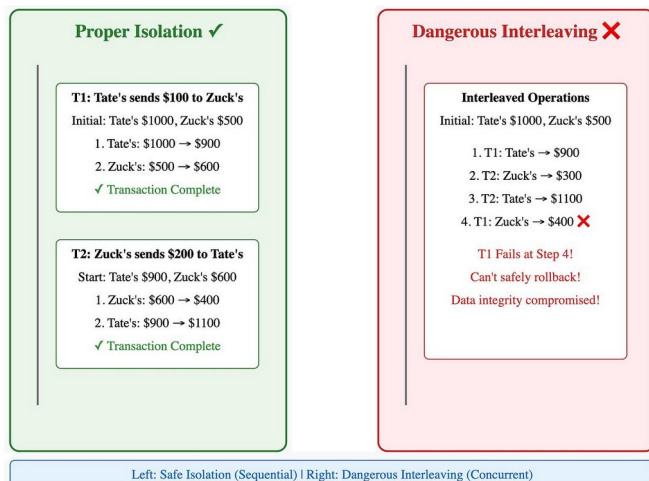
Easy Way to Remember

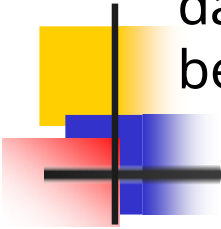
“Transactions should not disturb each other.”

Related Point: Cascading Rollback

Because uncommitted changes are hidden from others, if one transaction fails and rolls back, other transactions are not affected.

Transaction Isolation Example





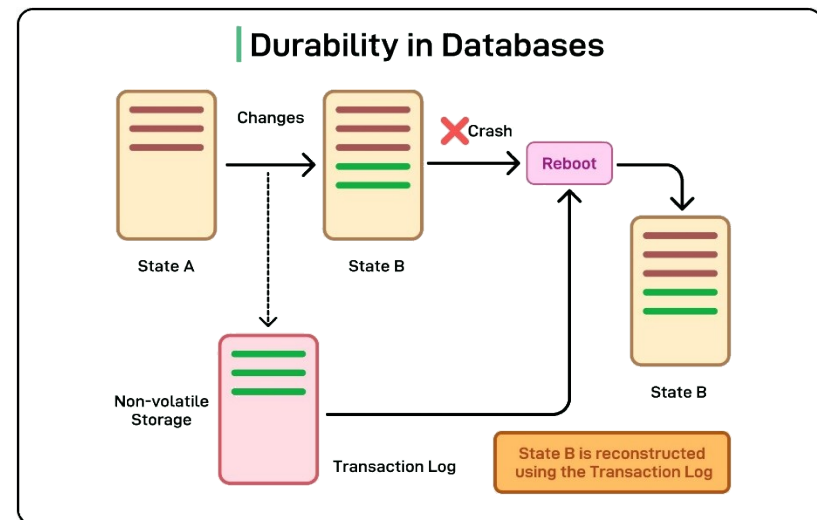
Durability or permanency: Once a transaction changes the database and the changes are committed, these changes must never be lost because of subsequent failure.

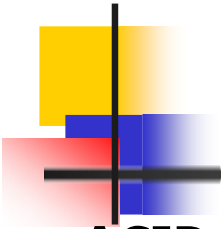
Flight Booking

You successfully book a flight and receive confirmation. Even if the airline's system crashes immediately afterward, your booking must still exist when the system recovers.

What Durability ensures

- Committed data survives power failures, crashes, and other problems.
- The database uses logs and backups to recover committed changes.





ACID in One Real-Life Example

ATM Cash Withdrawal

When you withdraw cash from an ATM:

1.Atomicity: Cash is dispensed and your balance is deducted together, or neither happens.

2.Consistency: Your account balance remains valid (cannot become incorrect unexpectedly).

3.Isolation: Other transactions cannot see your balance change until the withdrawal is completed.

Durability: Once the withdrawal is confirmed, the updated balance remains even if the ATM crashes later.

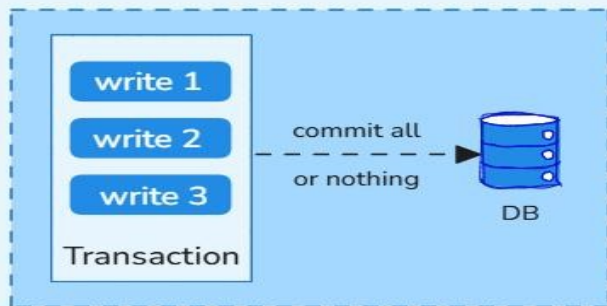
Remember ACID as:

- A → All or nothing
- C → Correct data
- I → Independent transactions
- D → Data stays forever

What does **ACID** mean?

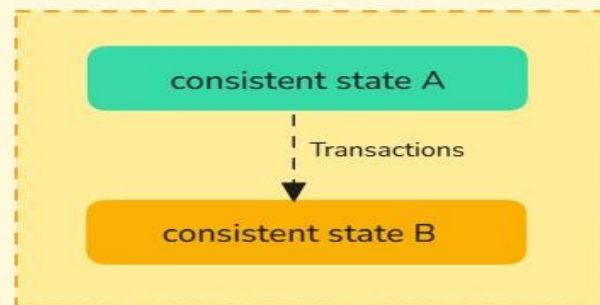
Atomicity

All or nothing



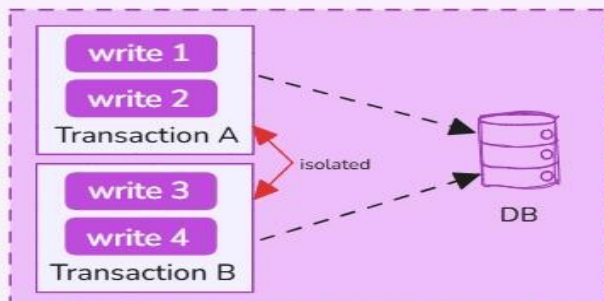
Consistency

Preserve database invariants



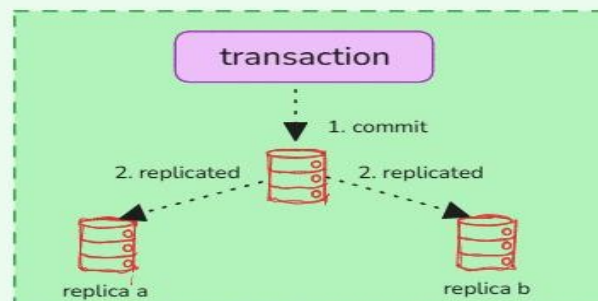
Isolation

Concurrent transactions are isolated from each other



Durability

Data is persisted after the transaction is committed even in a system failure

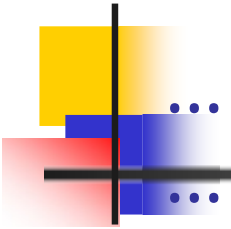




- Why recovery is needed

...

1. **A computer failure (system crash):** A hardware or software error occurs in the computer system during transaction execution. If the hardware crashes, the contents of the computer's internal memory may be lost.
2. **A transaction or system error :** Some operation in the transaction may cause it to fail, such as integer overflow or division by zero. Transaction failure may also occur because of erroneous parameter values or because of a logical programming error. In addition, the user may interrupt the transaction during its execution.

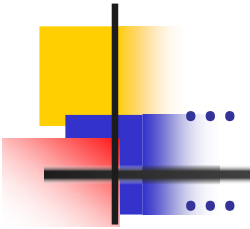


... - Why recovery is needed

3. **Local errors or exception conditions** detected by the transaction:

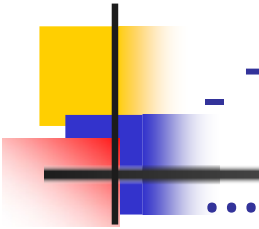
- certain conditions necessitate cancellation of the transaction. For example, data for the transaction may not be found. A condition, such as insufficient account balance in a banking database, may cause a transaction, such as a fund withdrawal from that account, to be canceled.
- a programmed abort in the transaction causes it to fail.

4. **Concurrency control enforcement:** The concurrency control method may decide to abort the transaction, to be restarted later, because it violates serializability or because several transactions are in a state of deadlock (see Chapter 18).



... - Why recovery is needed

5. **Disk failure:** Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash. This may happen during a read or a write operation of the transaction.
6. **Physical problems and catastrophes:** This refers to an endless list of problems that includes power or air-conditioning failure, fire, theft, sabotage, overwriting disks or tapes by mistake, and mounting of a wrong tape by the operator.

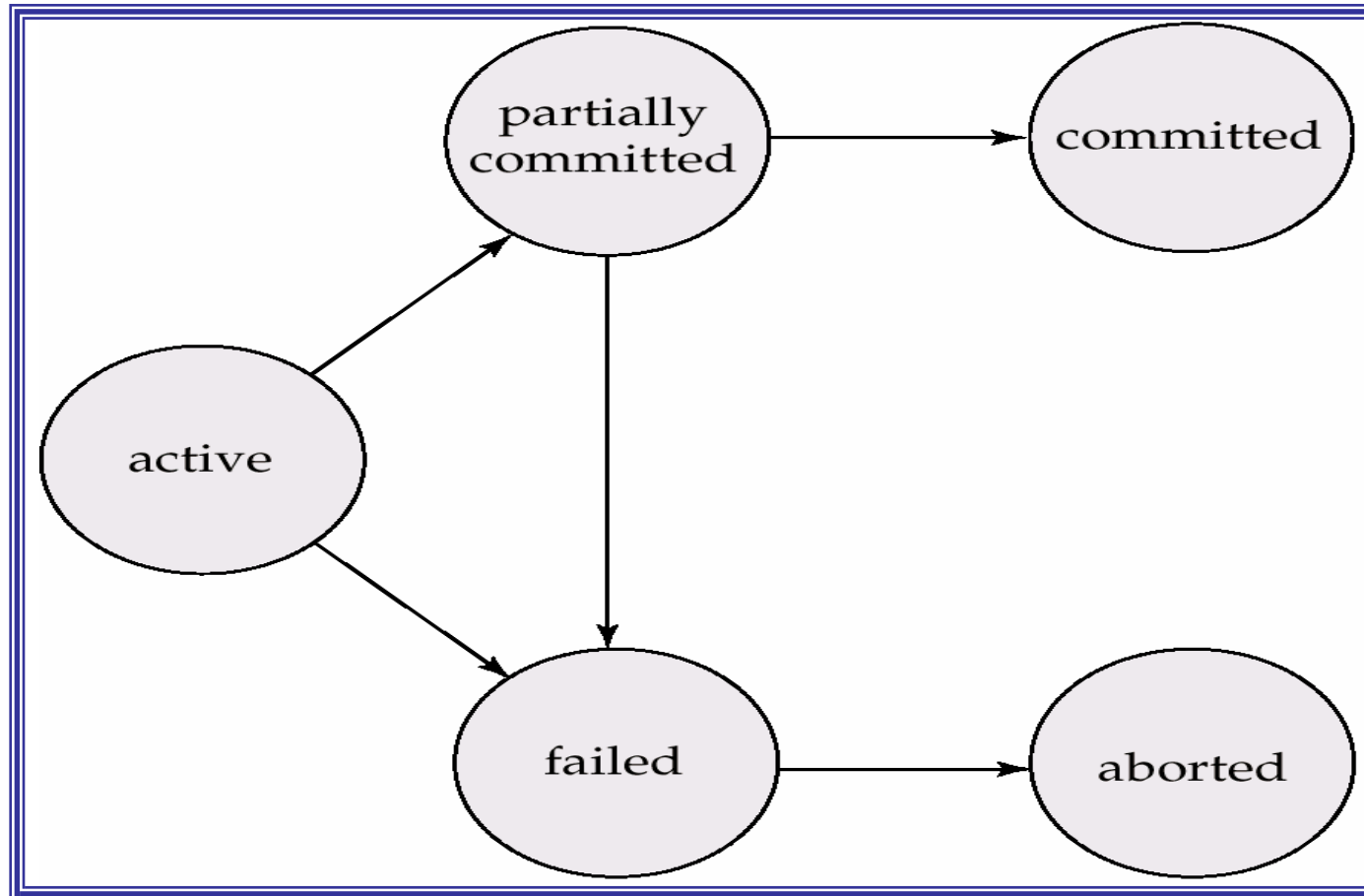


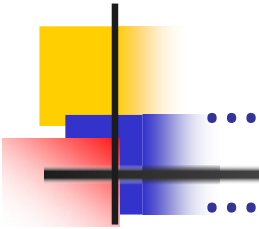
- Transaction and System Concepts

- A **transaction** is an atomic unit of work that is either completed in its entirety or not done at all. For recovery purposes, the system needs to keep track of when the transaction starts, terminates, and commits or aborts.

- **Transaction states:**
 - Active state
 - Partially committed state
 - Committed state
 - Failed state
 - Terminated State

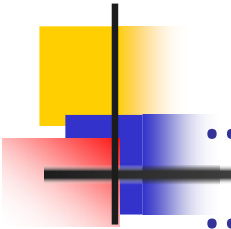
... - Transaction and System Concepts





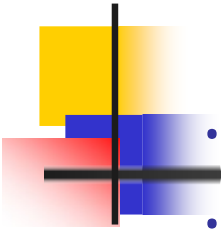
... - Transaction and System Concepts

- Recovery manager keeps track of the following operations
 - ...
 - **begin_transaction:** This marks the beginning of transaction execution.
 - **read or write:** These specify read or write operations on the database items that are executed as part of a transaction.
 - **end_transaction:** This specifies that read and write transaction operations have ended and marks the end limit of transaction execution. At this point it may be necessary to check whether the changes introduced by the transaction can be permanently applied to the database or whether the transaction has to be aborted because it violates concurrency control or for some other reason.



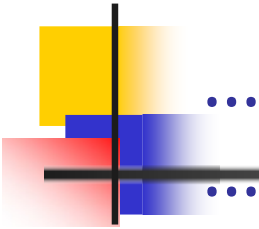
... - Transaction and System Concepts

- ... Recovery manager keeps track of the following operations
 - **commit_transaction**: This signals a *successful end* of the transaction so that any changes (updates) executed by the transaction can be safely **committed** to the database and will not be undone.
 - **rollback (or abort)**: This signals that the transaction has *ended unsuccessfully*, so that any changes or effects that the transaction may have applied to the database must be *undone*.



... - Transaction and System Concepts

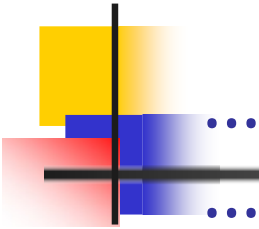
- Recovery techniques use the following operators:
 - **undo:** Similar to rollback except that it applies to a single operation rather than to a whole transaction.
 - **redo:** This specifies that certain *transaction operations* must be *redone* to ensure that all the operations of a committed transaction have been applied successfully to the database.



... - Transaction and System Concepts

- **The System Log ...**

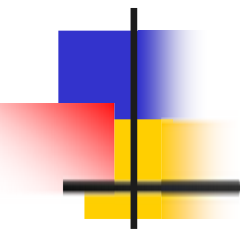
- **Log or Journal** : The log keeps track of all transaction operations that affect the values of database items. This information may be needed to permit recovery from transaction failures. The log is kept on disk, so it is not affected by any type of failure except for disk or catastrophic failure. In addition, the log is periodically backed up to archival storage (tape) to guard against such catastrophic failures.
- T in the following discussion refers to a unique **transaction-id** that is generated automatically by the system and is used to identify each transaction:



... - Transaction and System Concepts

■ ... The System Log - Types of log record:

1. **[start_transaction,T]:** Records that transaction T has started execution.
2. **[write_item,T,X,old_value,new_value]:** Records that transaction T has changed the value of database item X from old_value to new_value.
3. **[read_item,T,X]:** Records that transaction T has read the value of database item X.
4. **[commit,T]:** Records that transaction T has completed successfully, and affirms that its effect can be committed (recorded permanently) to the database.
5. **[abort,T]:** Records that transaction T has been aborted.



En
d